

Recap





Images VS Containers

- **An image** is a file-system bundle that contains all dependencies required to execute a process: files in the file system, installed packages, available resources, running processes, and kernel modules.
- **A Container** is an instance of image.
- **Containers** are **ephemeral** by default, which means that the container engine removes the writable layer when you remove the container.
- **Images are immutable**, which means they can't be modified once created. If changes need to be made to an image, you must **create a new image with the desired modifications**. In contrast, **containers are mutable and allow modifications during runtime**.
- **Container** is the **executable versions of the image** that include references to networking, disks, and other runtime necessities.



Registry

- **Registry** acts as a **place to store container images** and share them out via a process of **uploading to (pushing) and downloading from (pulling)**.
- **Registry** is a **collection of repositories**, while **repositories** hold a **collection of related images**.

Types of registries

- **Public Registry:** Docker Hub
- **Private Registry :** Amazon Elastic Container Registry (ECR), Red Hat Quay (will be used in this course).

Chapter 2.1 Podman Basics



Outline



- Run a containerized service with Podman.
- Describe how containers communicate with each other.
- Expose ports to access containerized services.
- Explore running containers.
- List, stop, and delete containers with Podman.



Introducing Podman

- Podman is an **open source tool** that you can use to manage your containers **locally**.
- Podman is **daemonless**.
- A daemon is a **process** that is always running and ready for receiving incoming requests.
- Podman interacts directly with containers, images, and registries without a daemon.
- Some other container tools use a **daemon** to proxy the requests, which brings a **single point of failure**.
- In addition, a daemon might require **elevated privileges**, which is a **security concern**.



Introducing Podman

- Podman comes in the form of a command-line interface (CLI), which is supported for several operating systems.
- Along with the CLI, Podman provides **two additional ways** to interact with your containers and automate processes:
 - 1) RESTful API
 - 2) desktop application called **Podman Desktop**.



Pulling and Displaying Images

Podman version

- Podman -v

Pull Images

- podman pull <image name>
- EX:
 - **Public Registry:** podman pull registry.access.redhat.com/ubi9/ubi-minimal:9.5
 - **Private Registry:** podman pull registry.redhat.io/rhel7/rhel:7.9
 - If you will download from a private registry, you need to login to the registry first.
 - podman login <registry-name>

Display Images

- podman images = podman image ls



Run Container

- `podman run --name <container name> <image name> <command>`
- `podman run <image name> <command>`
 - `podman run registry.access.redhat.com/ubi9/ubi-minimal:9.5 echo 'Red Hat'`
 - `podman run --rm registry.access.redhat.com/ubi9/ubi-minimal:9.5 echo 'Red Hat'`
- if a container name is not provided during the creation of a container, then Podman **generates a random string name for the container**. It is important to define a unique name to facilitate the identification of your containers when managing their lifecycle.
- if you run a container from an image that is not stored in your system then **Podman tries to pull the image before running the container. Therefore, it is not necessary to execute the pull command first.**
- Images can exist without containers, but containers are dependent of images because containers use container images to build a runtime environment to execute applications.



Displaying Containers

- `podman ps` → list all running containers
- `podman ps --all` → list all containers (running and stopped)
- `podman ps -a` → list all containers (running and stopped)
- `podman ps --all --format=json`



Exposing Containers

- `podman run -p host:container <image name>`
 - `podman run -p 8081:8080 registry.access.redhat.com/ubi8/httpd-24:latest`
- This command creates a new container that runs an Apache HTTP server in foreground by mapping the 8081 port in your local machine to the 8080 port inside the container.
- If you want the container **to run in the background**, to avoid the terminal being blocked, you can use the `-d` option.
- `podman run -d -p host:container <image name>`
- You can access the HTTP server :
 - 1) `curl http://localhost:host port`
 - 2) Using firefox: localhost:host port



Using Environment Variables

- `podman run -e NAME='Red Hat' <image name> printenv NAME`
 - `podman run -e NAME='Red Hat' registry.redhat.io/rhel7/rhel:7.9 printenv NAME`
 - Podman ps -a ?? why the container exits ?
- You can pass environment variables to a container by using the `-e` option. In the following example, an environment variable called `NAME` with the value `Red Hat` is passed. Then, the environment variable is printed by using the `printenv` command inside the container.

Lab: section 2.2



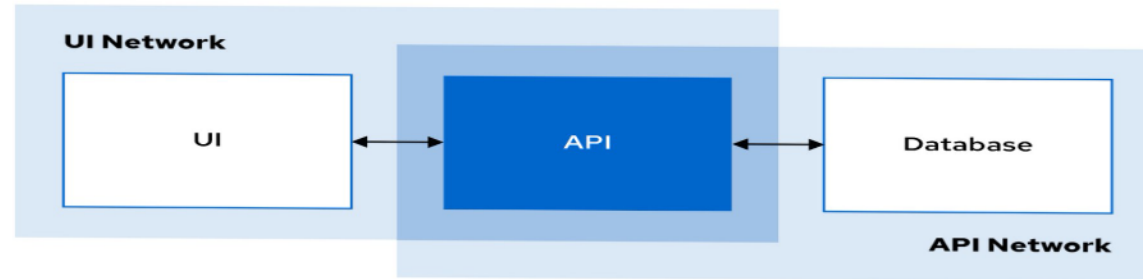
Chapter2: Section2.3





Container Networking Basics

- Podman comes with a **network** called **podman**. By default, containers are attached to this network and can use it to communicate with one another.
- For example, the containers running an **application API and database** can use a separate Podman network to isolate their communication from other containers. Similarly, that same **API container** can **use yet another network** to isolate communication with a third container that hosts the application UI. (API & UI)
- So, Containers can be connected to one or more Podman networks.





Managing Podman Networks

- Podman network management is done via the podman network subcommand. These management operations include:
- **podman network create <network-name>**: creates a new Podman network.
- **podman network ls**: lists existing networks and a brief summary of each.
- **podman run -d --name <container-name> --net <network-name1,network-name2> <image-name>**: creates a new container which is connected to multiple network separated by comma.
- **podman network connect <network-name> <container-name>**: connects an already running container to or from an existing network.



Managing Podman Networks

- `podman run -d --name my-container nginx`
 - `podman inspect my-container | grep -i network`
"NetworkSettings": {
 "NetworkMode": "slirp4netns",
- If a network is not specified with the podman run command, then the container connects to the default network.
- The default network uses the **slirp4netns** network mode, **because it cannot attach to CNI networks like podman**, without root privileges (they require access to system network interfaces) and the networks that you create with the podman network create command use the bridge network mode.
- If you try to connect a bridge network to a container by using the slirp4netns network mode, then the command fails.



Rootful and Rootless Container Networks

- Podman networking behaves differently when you run containers as the root user, rootful containers, compared to when you run containers as a non-root user, rootless containers.
- When you run rootful containers, then Podman uses the **podman network which is pre-configured and attaches containers to it.**
- Note, that the podman default network comes with DNS disabled by default. Without DNS, containers attached to a network can reach other containers by using the container's IP address, but they cannot use container names to communicate with each other.
- If you run a rootless container, then the non-root user cannot create networking bridges or manage virtual interfaces on the host.
- Instead, Podman creates the required networking interfaces on a network namespace. By default, Podman does not attach rootless containers to the podman default network.
- If you want rootless containers to communicate with each other, then you must create a Podman network and attach the containers to it, or attach the containers to the default Podman network.



Managing Podman Networks

- **podman network rm <network-name>**: removes a network.
- **podman network prune** : removes any networks that are not currently in use by any running containers.

Enabling Domain Name Resolution

- When using the default network, **the domain name system (DNS) is disabled** and can only be enabled **by overwriting the default network**.
- **To use DNS**, create a new Podman network and connect your containers to that network.
- When using a network with DNS enabled, a container's hostname is the name assigned to the container.
- Other Containers on the same network can make requests to container using the hostname (container-name). **This means that the container name resolves to the current IP address of the container.**



Enabling Domain Name Resolution

- **Create a new network.**
 - `podman network create my-net`
- **Check if the new network is enabled.**
 - `podman network inspect my-net | grep -i dns`
 `"domainName": "dns.podman",`
 `"type": "dnsname"`
 - This means **DNS is enabled**.
- **Create two containers and attach them to the new network.**
 - `podman run -d --name cont1 --net my-net nginx`
 - `podman run -d --name cont2 --net my-net nginx`
 - `podman exec -it cont1 /bin/bash`
 - `Apt install curl`
 - `Curl http://cont2`
 - If you want to another container with different network to talk to cont1, so you should curl with container ip address as dns is not enabled.

Try cont 3 and
cont 4 comm
with ip address

Section 2.5: Accessing Containerized Network Services





Accessing Containerized Network Services

- **podman run -p host-port:container-port <container-name>** : map port 80 inside the container to port 8075 on the host machine.
- **podman run -p host-ip:host-port:container-port <container-name>** : publish a container to a specific host and to limit the networks (127.0.0.1 same as localhost)
- **podman port <container-name>** : list port mappings for a container (output: <container-port>/tcp → <host-ip>:<host-port>)
- **podman port -- all** : lists port mappings for all containers.
- **podman inspect <container-name> -f '{{.NetworkSettings.Networks.<network-name>.IPAddress}}'** : retrieves the private IP address of the container within the specific network.

Note: IP address is **only valid** within the this network.

Section 2.7: Accessing Containers

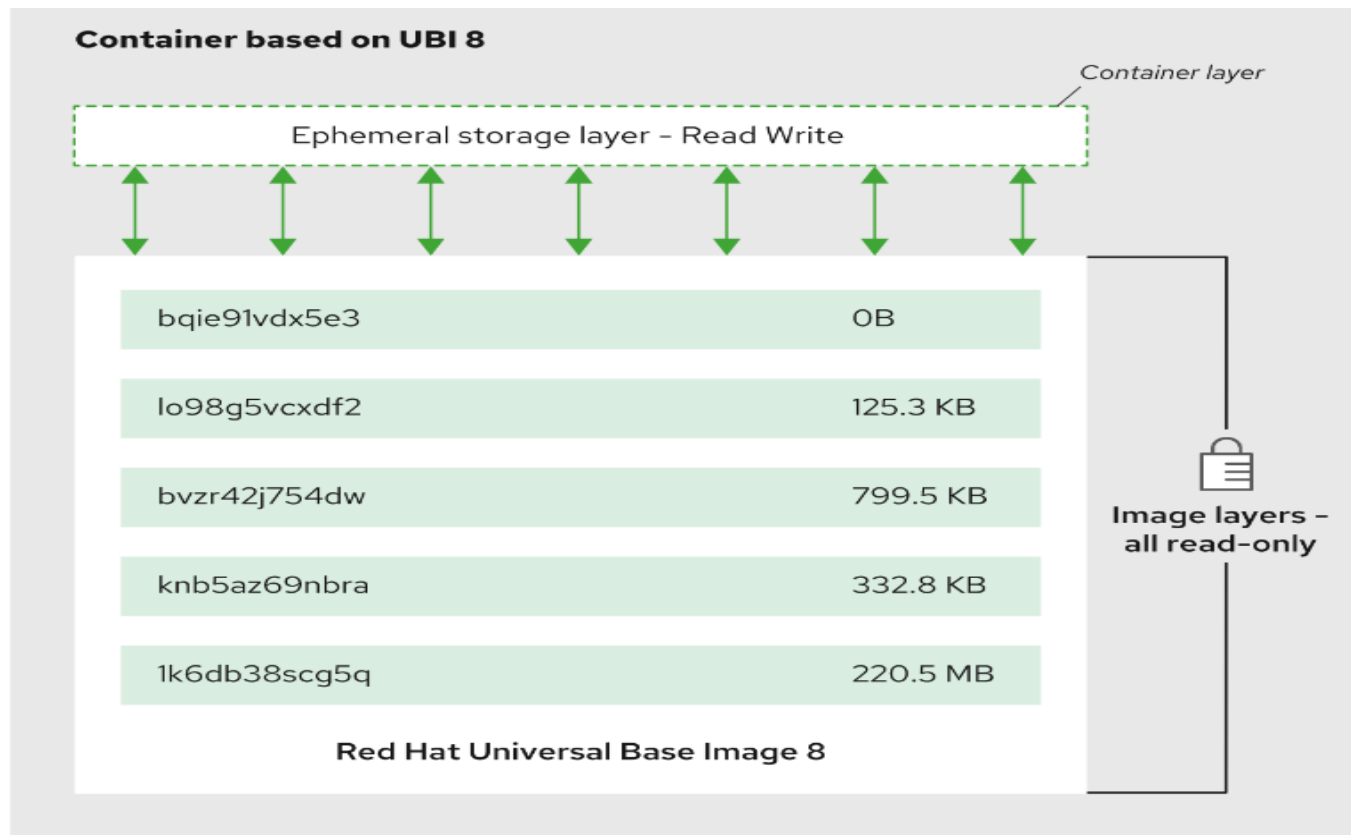




Container Layers

- Container images are characterized as **immutable** and **layered**.
- Each image layer consists of a set of file system differences. A diff signals a file system change from the previous layer.
- When you start a container, the container uses a new **ephemeral layer over its base container image layers**.
- This layer is the **only read/write** storage available for the container by default, and it is used for any runtime file system operations, such as creating working files, temporary files, and log files.
- Those files are considered **volatile, which means the files are deleted when you delete the container ?**
- The container storage layer is exclusive to the running container, so if you create another container from the same base image, the new container creates another read/write layer. **This ensures the each container's runtime data are isolated from other containers.**

Container Layers





Start Processes in Containers

- **podman exec [option] <container-name> <command>** : start a new process in a running container.
[option]: Use --env or -e to specify environment variables.
Use --interactive or -i to instruct the container to accept input.
Use --tty or -t to allocate a pseudo terminal.

Copy Files In and Out of Containers

- **podman cp [options][container]:source_path [container]:destination_path** : copy files to and from a running container.
 - podman cp a3bd6c81092e:/tmp/logs .
 - podman cp nginx.conf nginx:/etc/nginx
 - podman cp nginx-test:/etc/nginx/nginx.conf nginx-proxy:/etc/nginx

Section 2.9: Managing the Container Lifecycle





Managing the Container Lifecycle

- **podman <stop/kill/pause/unpause> <container-name>**
- Both podman **stop** and podman **kill** eventually send a **SIGKILL** signal to the container.
- The podman **pause** command suspends all processes in the container by sending the **SIGSTOP** signal.
- The podman **unpause** command **unsuspends** a paused container.
- The podman **restart** command to start a stopped container again. If the command is applied to a **running container**, then the container is also restarted.
- The podman **rm** command to remove a stopped container. You can use **--force flag** to remove the container forcefully (if it is not stopped).

You can combine the **--force** and **--all** flags to remove all containers, including running containers.

Lab: section 2.11



Podman VS Docker



Feature	Docker	Podman
Architecture	Client–Server model (uses a central daemon: dockerd)	Daemonless (no central service)
Daemon	Requires a background daemon (dockerd) to manage containers	No daemon; each Podman command manages containers directly
User Privileges	Typically runs with root privileges (unless configured otherwise)	Supports rootless containers — can run as a normal user
Process Management	Containers are child processes of the Docker daemon	Containers are child processes of the Podman command (no middle layer)
Dependency	If the Docker daemon stops, all containers stop	Containers continue running independently (no daemon dependency)
Security	Higher risk since daemon runs as root	More secure — no root daemon required
Command Compatibility	Uses docker commands	Commands are almost identical (alias docker=podman works)
Single Point of Failure	Yes — the daemon	No — each container is independent
Architecture	[User CLI] → [Docker Client] → [Docker Daemon] → [Containers]	[User CLI] → [Podman] → [Containers]